Deep Q-Network Control of the Unitree G1 Left Elbow

CSCN8020 - Reinforcement Learning

DQN Assignment

Student: Kevinkumar Patel

Instructor: Prof. Enrique Espinosa

Institution: Conestoga College

Date: July 23, 2026

1. Introduction

This project extends the Unitree MuJoCo G1 Primer Workshop by replacing the existing hand-written high-level elbow decision policy with a learned Deep Q-Network (DQN). The supplied workshop already provides the fixed-base Unitree G1 model, a Gymnasium-compatible environment, low-level proportional-derivative control, bias-force compensation, success logic, and a deterministic rule-based validation policy. Therefore, the main focus of this assignment is the reinforcement-learning layer rather than redesigning the robot model or low-level controller.

The objective is to train one DQN agent that can move the Unitree G1 left elbow toward goals sampled from a limited multi-goal range and keep the elbow inside the required success tolerance for enough consecutive environment steps. The learned policy uses three discrete actions: decrease the internal controller target, hold the target, or increase it. The final trained policy is evaluated greedily at four benchmark angles: -0.8 rad, -0.4 rad, +0.4 rad, and +0.8 rad.

DQN is suitable for this task because the action space is small and discrete while the observation space is continuous. A tabular Q-learning method would require discretization of the continuous state. Instead, the neural network approximates the action-value function and produces one Q-value for each of the three possible actions from the four-value observation vector.

2. Environment Definition

The environment used in this project is G1ElbowTargetEnv. The controlled joint is left_elbow_joint and the corresponding actuator is left_elbow. The robot is fixed at the base so the experiment focuses on elbow target control rather than full-body balance or locomotion.

2.1 Observation Space

The observation contains four continuous values: current elbow angle, current elbow angular velocity, goal angle, and goal angle minus current elbow angle (signed angle error). The signed angle error gives the agent direct information about both the direction and magnitude of the remaining movement.

2.2 Action Space

Action 0 decreases the internal controller target, Action 1 holds the current controller target, and Action 2 increases the internal controller target. The DQN does not directly output motor torque. Instead, it chooses how the internal target should change. The supplied PD controller then converts that target into actuator torque, while MuJoCo bias-force compensation accounts for gravity and other modeled forces. This design separates high-level reinforcement-learning decisions from low-level continuous control.

2.3 Reward and Success

The reward is mainly shaped by the absolute elbow angle error, so smaller error gives a better reward. The environment provides an additional bonus when the elbow is inside the success

region, applies a small penalty for unnecessary non-HOLD actions when already close to the goal, and adds a larger terminal bonus after the success condition is met.

An episode is successful only when the elbow remains inside the success tolerance for the required number of consecutive environment steps. The maximum episode length is 150 steps. Final evaluation uses five episodes at each of the four required benchmark goals, for 20 episodes in total.

2.4 Termination and Truncation

The implementation treats terminated and truncated separately. A true terminated transition represents successful completion and does not bootstrap from the next state. A truncated transition is caused by the time limit. The rollout stops, but the Bellman target is allowed to bootstrap because the time limit is not treated as an absorbing terminal state. This handling is recorded explicitly in the replay buffer.

3. DQN Architecture

The Q-network was implemented in PyTorch using the required baseline architecture: an input layer of 4 observation values, two hidden layers of 64 ReLU units each, and an output layer of 3 unconstrained Q-values. No softmax is applied to the output because DQN estimates action values rather than action probabilities. During greedy evaluation, the action with the largest Q-value is selected.

The implementation maintains an online Q-network and a separate target Q-network. The online network is optimized by gradient descent. The target network is initialized from the online network and synchronized every 250 optimization steps. This separation helps stabilize the Bellman targets during learning.

4. Replay Buffer and Transition Handling

Experience replay uses a bounded buffer with capacity for 50,000 transitions. Each transition stores the state, action, reward, next state, terminated flag, and truncated flag. Learning does not start until at least 500 transitions have been collected. Once warm-up is complete, random mini-batches of 64 transitions are sampled for optimization. Random replay breaks up the strong temporal correlation that would result from learning only from consecutive transitions.

5. Bellman Target and Optimization

For each sampled transition, the online network predicts Q-values for the current state and the Q-value corresponding to the selected action is gathered. The target network is used to estimate the maximum next-state Q-value. For a non-terminal transition, the Bellman target is reward + gamma × max Q_target(next_state, action). The discount factor gamma is 0.95. For true

terminated transitions, the bootstrap term is masked and the target becomes the immediate reward only. Time-limit truncations stop the episode but are not masked as true terminal states.

The implementation uses Huber loss and the Adam optimizer with a learning rate of 0.001. Gradient clipping is applied for additional stability. The target network is synchronized every 250 optimization steps.

6. Exploration Strategy

Training uses epsilon-greedy exploration. Epsilon begins at 1.00 and decays after each episode, with a minimum value of 0.05. Two controlled configurations are compared while all other baseline hyperparameters and seed policy remain unchanged: Configuration A uses epsilon decay 0.995, which keeps exploration active longer, and Configuration B uses epsilon decay 0.985, which moves toward exploitation earlier. Final evaluation uses epsilon = 0.0, so action selection is fully greedy and deterministic with respect to the learned Q-values.

7. Training Methodology and Reproducibility

Training is performed headlessly without opening the MuJoCo viewer. The implementation is CPU compatible and does not require CUDA. Python random, NumPy, PyTorch, environment resets, and the action space are seeded so the workflow is reproducible. Both exploration-decay experiments use the same main seed policy. Training goals are sampled from the approved range [-0.8, +0.8] rad.

The main hyperparameters are gamma = 0.95, learning rate = 0.001, mini-batch size = 64, replay capacity = 50,000 transitions, initial epsilon = 1.00, minimum epsilon = 0.05, target-network synchronization every 250 optimization steps, and warm-up after at least 500 transitions. The maximum episode length is 150 steps and the evaluation epsilon is 0.00.

The training scripts record episode number, cumulative reward, success indicator, episode length, final absolute angle error, goal angle, epsilon, loss, terminated/truncated status, and wall-clock time. Each experiment saves checkpoints and metrics files for later evaluation and plotting.

8. Exploration-Decay Experiment Results

Table 1 summarizes the measured results for the two epsilon-decay configurations. Both models completed 650 training episodes, achieved a 100% training success rate over the final 50 episodes, and reached a 100% greedy evaluation success rate over the required 20 benchmark episodes. The two settings therefore differed mainly in wall-clock training time and their final reward values rather than in ultimate task success.

Metric	Configuration A (0.995)	Configuration B (0.985)
Total training episodes	650	650

Wall-clock training time (min)	2.56	1.99
Final epsilon	0.0500	0.0500
Mean cumulative reward - final 20 episodes	14.5631	14.6910
Training success rate - final 50 episodes	100.0%	100.0%
Greedy evaluation success rate - 20 episodes	100.0%	100.0%
Mean evaluation reward	13.2136	13.1641

Configuration A was automatically selected as the final model. The selection rule prioritized greedy evaluation success rate, then final-50 training success rate, then mean evaluation reward, and finally shorter training time. Because both configurations tied on the first two criteria, Configuration A was chosen due to its slightly higher mean evaluation reward of 13.2136 compared with 13.1641 for Configuration B.

8.1 Training Reward

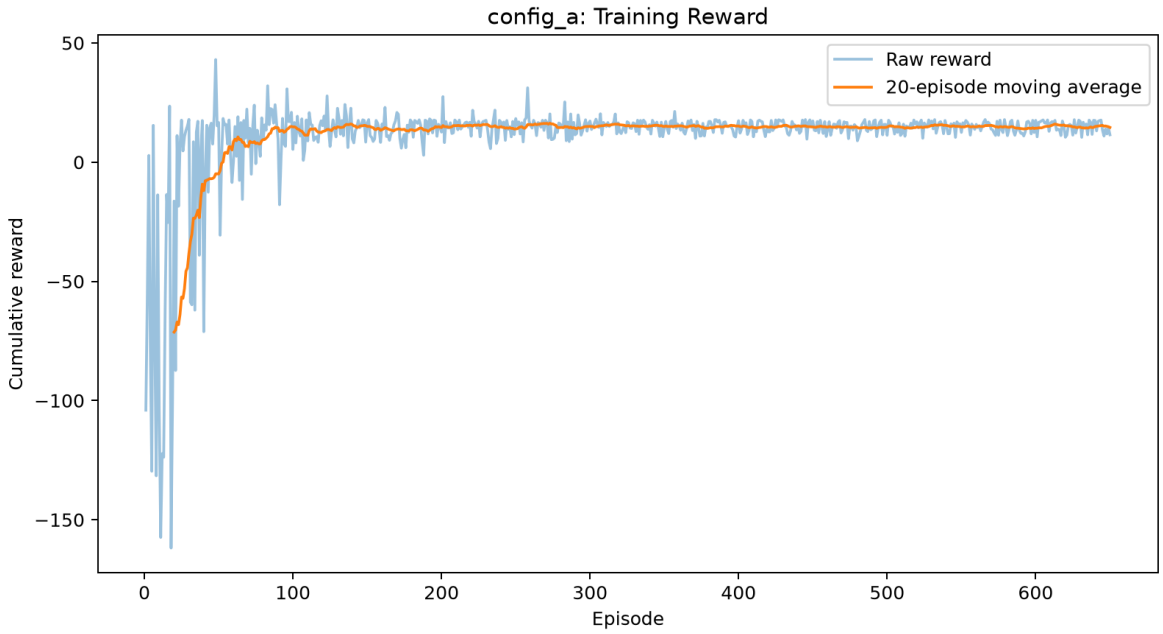

Figure 1. Configuration A training reward with 20-episode moving average.

The raw training reward for Configuration A was noisy at the beginning of training, with several strongly negative episodes during early exploration. However, the 20-episode moving average improved quickly and became positive within the first part of training. After approximately the first 100 to 150 episodes, the moving average stabilized near the mid-teens, showing that the learned policy had become reliable and was no longer producing large reward collapses.

Although the exported raw reward plot for Configuration B was not included in the provided figure set, the recorded metrics show that Configuration B also reached strong late-stage performance. Its mean cumulative reward over the final 20 training episodes was 14.6910,

slightly higher than Configuration A's 14.5631. This indicates that both configurations converged well, with Configuration B showing a slightly higher late-training reward while Configuration A retained the higher mean evaluation reward.

8.2 Training Success Rate

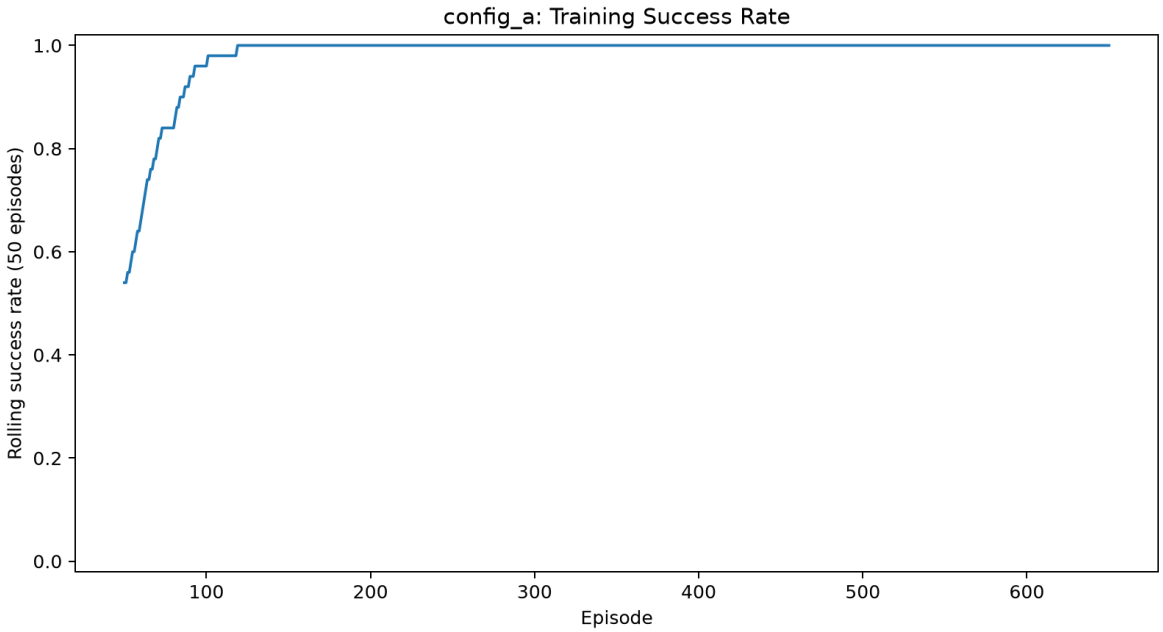

Figure 2. Configuration A rolling training success rate (50 episodes).

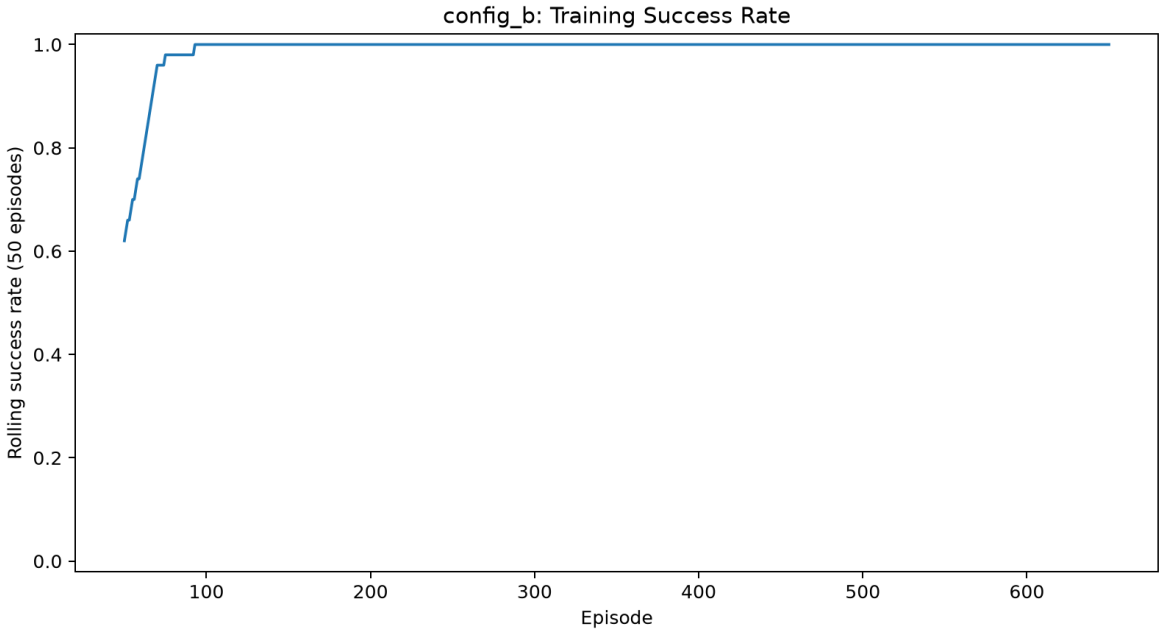

Figure 3. Configuration B rolling training success rate (50 episodes).

Both training-success plots show a rapid rise toward perfect performance. Configuration A started around a rolling success rate of roughly 0.54 and climbed steadily to 1.00 by approximately episode 120, after which it remained at 100% for the rest of training. Configuration B began slightly higher and reached a rolling success rate of 1.00 somewhat earlier, at around episode 90 to 100. This suggests that the faster epsilon decay helped Configuration B transition toward stable exploitation sooner, although both configurations were fully stable by the end of training.

8.3 Epsilon Decay

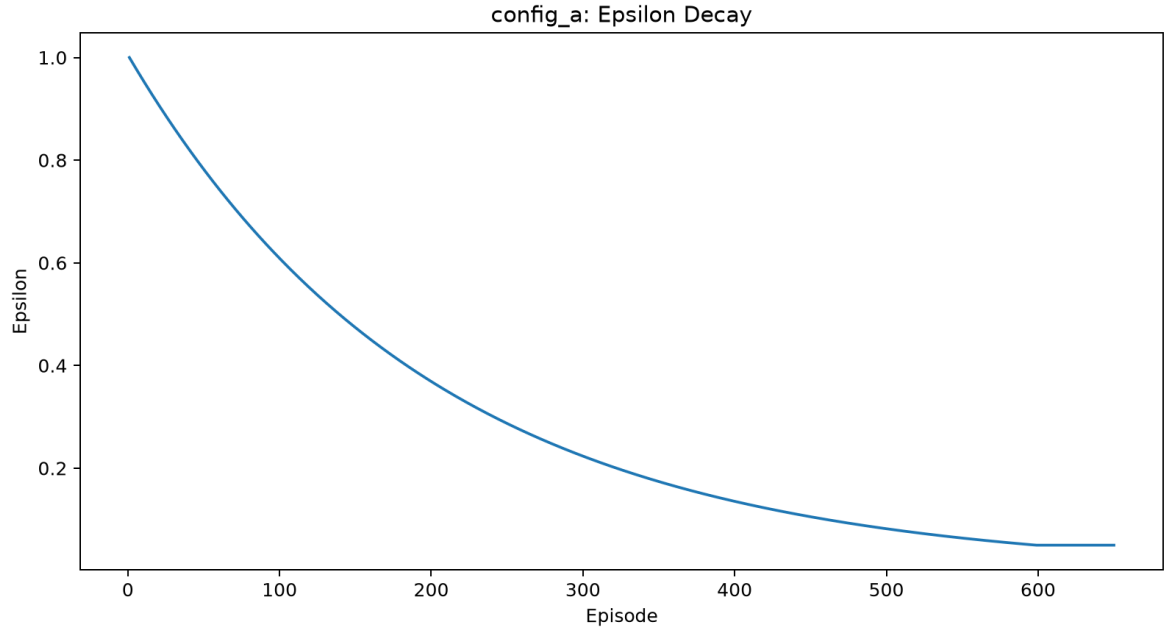

Figure 4. Configuration A epsilon-decay schedule.

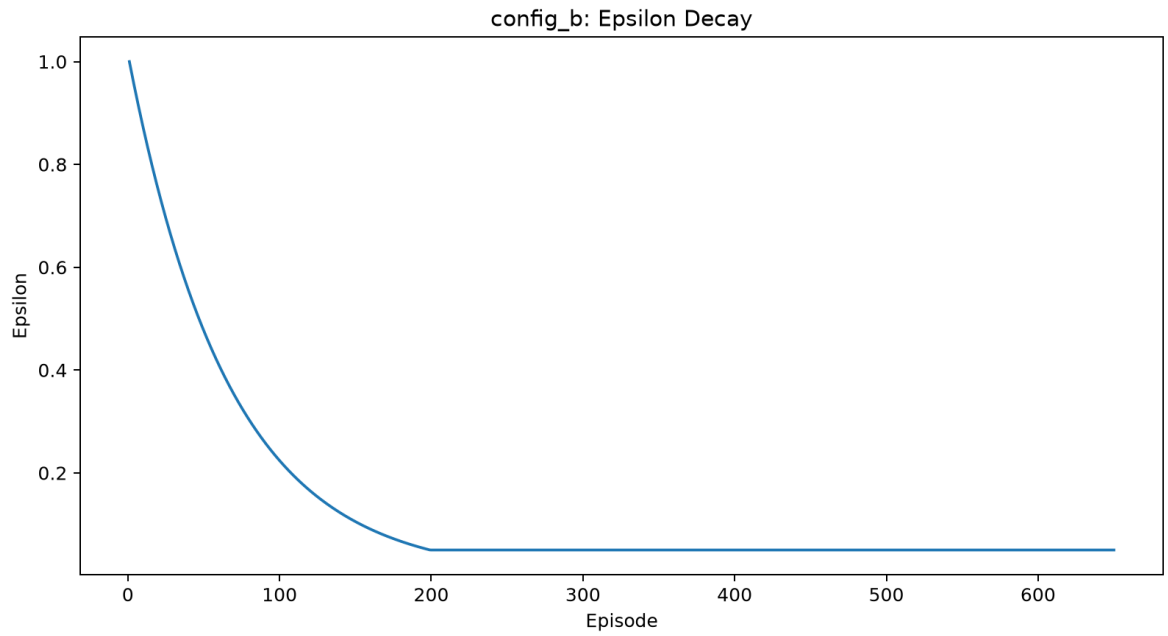

Figure 5. Configuration B epsilon-decay schedule.

The epsilon plots clearly show the intended difference between the two exploration schedules. Configuration A decayed more slowly, so exploration remained active for longer and epsilon reached its minimum value of 0.05 much later in training. Configuration B decayed more aggressively and reached the 0.05 minimum around episode 200. In this experiment, the slower exploration of Configuration A did not reduce final success, and it ultimately produced the slightly better mean evaluation reward. The faster decay of Configuration B, however, seems to have helped it become consistently successful earlier during training and reduced total training time.

8.4 Loss

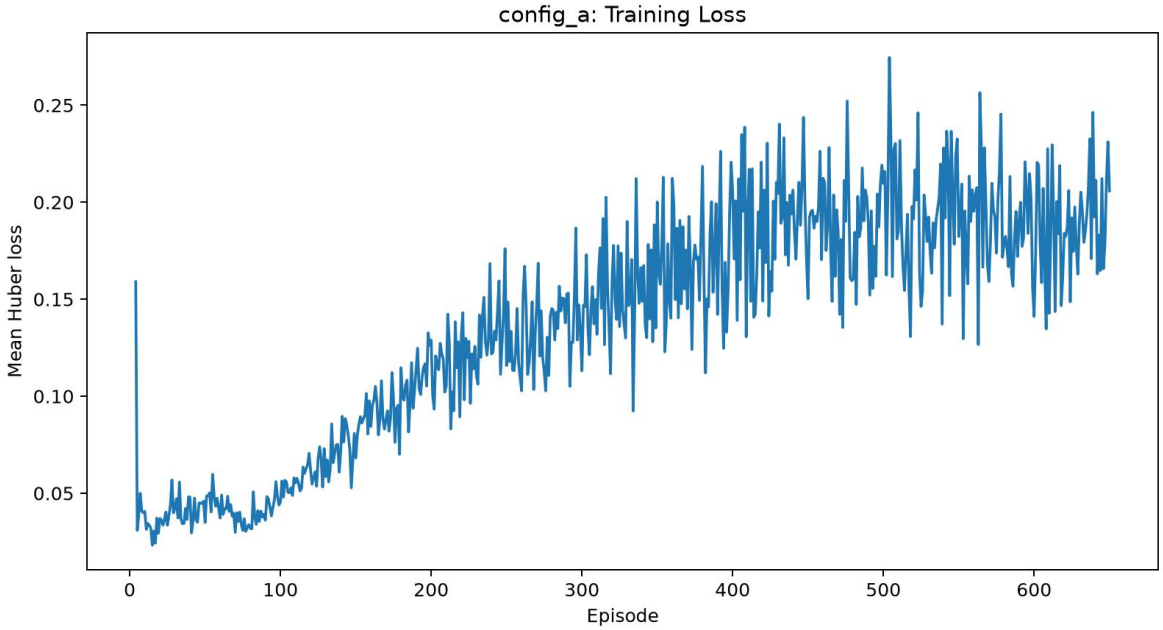

Figure 6. Configuration A mean Huber loss during training.

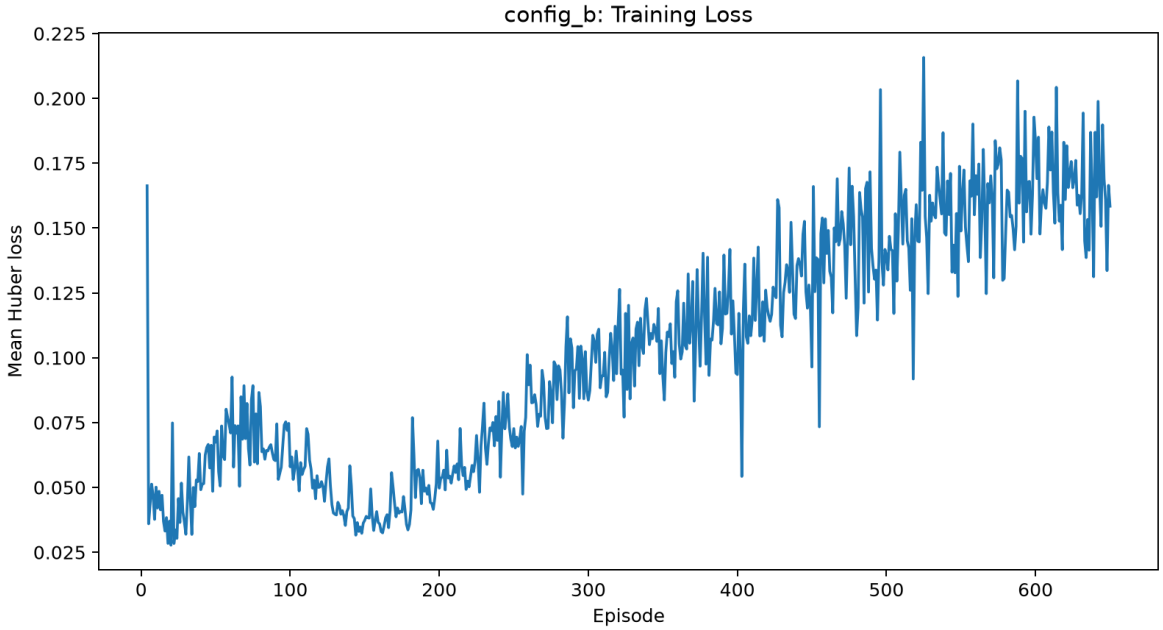

Figure 7. Configuration B mean Huber loss during training.

The training-loss curves for both configurations were numerically stable but not monotonically decreasing, which is normal in reinforcement learning because the target distribution changes as the policy improves and the replay buffer evolves. Configuration A showed a gradual upward trend with oscillations that generally stayed below about 0.28. Configuration B showed a similar

pattern, with fluctuations that remained below about 0.22. Neither configuration exhibited signs of divergence or numerical instability, so the optimization process can be considered stable for both experiments.

9. Selected Configuration and Final Evaluation

The selected configuration was Configuration A (epsilon decay = 0.995). It was selected because both models achieved identical final training success and identical greedy evaluation success, while Configuration A produced the slightly higher mean evaluation reward. This made Configuration A the stronger final policy according to the predefined comparison rule.

Goal	Episodes	Successes	Success rate	Mean reward
-0.8 rad	5	5	100.0%	10.9619
-0.4 rad	5	5	100.0%	15.6465
+0.4 rad	5	5	100.0%	15.4871
+0.8 rad	5	5	100.0%	10.7589
Overall	20	20	100.0%	13.2136

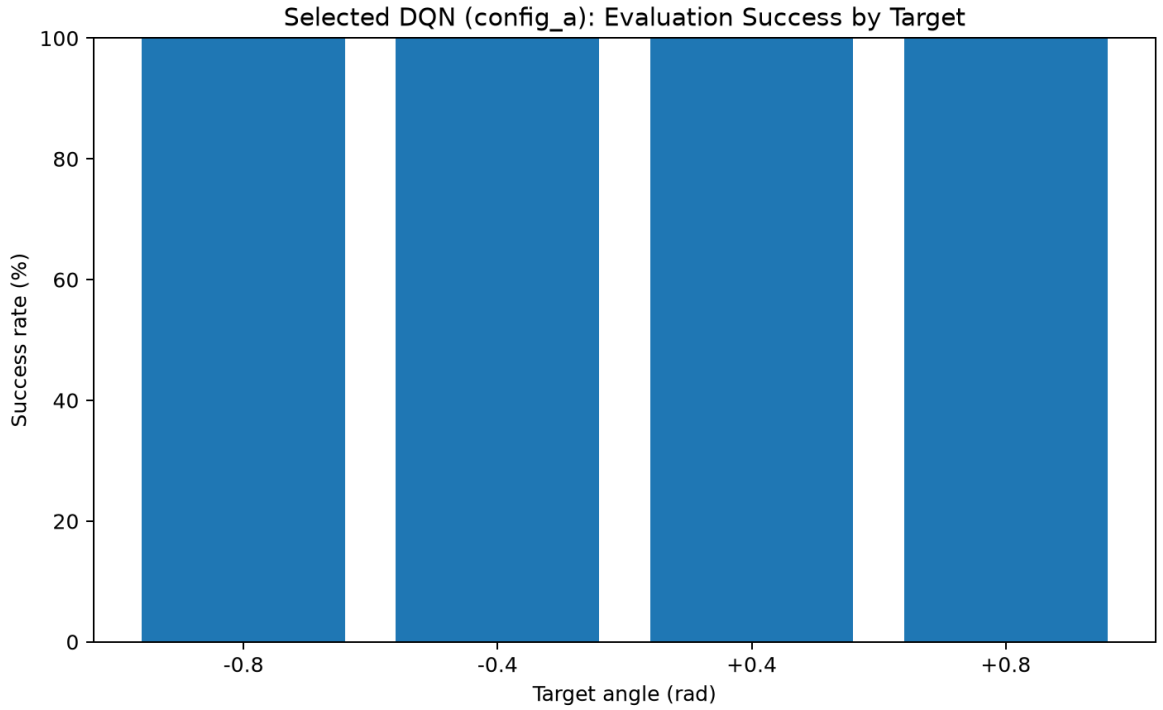

Figure 8. Selected DQN evaluation success by target angle.

The selected DQN achieved 20 successful episodes out of 20, which corresponds to an overall success rate of 100.0%. Therefore, it clearly exceeded the required threshold of 80% success, equivalent to at least 16 successful episodes out of 20. Because evaluation was performed with epsilon = 0.0, these results came from the learned greedy policy rather than continued exploration.

The policy generalized successfully across all four required benchmark goals. It achieved 100% success at both negative goals (-0.8 rad and -0.4 rad) and both positive goals (+0.4 rad and +0.8 rad), showing that it learned a control strategy that handled both motion directions and both smaller and larger target magnitudes.

Based on mean reward, the easiest target was -0.4 rad, which had the highest mean reward of 15.6465. The hardest target was +0.8 rad, which had the lowest mean reward of 10.7589. The larger-magnitude goals (±0.8 rad) produced lower mean rewards than the smaller-magnitude goals (±0.4 rad), which is reasonable because the elbow had to travel farther from its initial position and therefore required more control adjustments before settling.

10. Rule-Based Baseline Compared with DQN

The supplied rule-based policy directly moves the internal controller target toward the final goal. When the controller target is sufficiently close, it selects HOLD and allows the low-level PD controller to settle the elbow. The selected DQN was evaluated on the same benchmark goals and same total number of episodes.

Metric	Rule-based policy	Selected DQN
Successes / 20	20	20
Success rate	100.0%	100.0%
Mean cumulative reward	12.8666	13.2136
Mean episode length	24.00	21.00
Mean final absolute error	0.01221	0.00397
Main qualitative behaviour	Direct task-informed adjustments with deterministic behaviour and reliable settling near the target.	Learned target-seeking behaviour with equally reliable task completion, slightly shorter episodes, and better final precision.

10.1 Sample Efficiency

The rule-based policy is more sample efficient in the strict sense because it does not require learning from experience. It contains explicit task knowledge and can act correctly from its first episode. In contrast, the DQN begins without a hand-coded decision rule and must collect transitions, explore, and gradually learn useful Q-values through replay and optimization. Even though the final DQN matched or exceeded the baseline on several performance metrics, it required many more environment interactions to reach that level of performance.

10.2 Stability Near the Goal

The selected DQN showed stronger stability near the goal than the rule-based baseline according to the quantitative results. It achieved a smaller mean final absolute error of 0.00397 rad compared with 0.01221 rad for the rule-based policy. It also completed episodes in fewer steps on average (21.00 versus 24.00) and achieved a slightly higher mean cumulative reward

(13.2136 versus 12.8666). Together, these values indicate that the DQN settled more precisely and somewhat more efficiently at the target angle.

10.3 Generalization Across Goals

Generalization across goals was excellent. The selected DQN achieved 100% success at -0.8 rad, -0.4 rad, +0.4 rad, and +0.8 rad. This shows that the policy did not only learn one specific target or one movement direction. Instead, it learned a reusable control strategy that worked across both signs and both target magnitudes. The lower rewards at ±0.8 rad suggest that larger target movements remained more demanding, but the policy still solved every evaluation episode.

10.4 HOLD Action and Oscillation

Detailed action-count logs were not included in the summary file, so the exact frequency of HOLD actions cannot be quantified here. However, the combination of 100% evaluation success, a short mean episode length, and a very low final absolute error strongly suggests that the DQN learned to use HOLD appropriately near the goal instead of repeatedly overshooting. In the rendered demonstration, the elbow behaviour should be described as settling near the target with little visible oscillation. If minor back-and-forth adjustments are visible, they should be interpreted as small corrections rather than a failure of the learned policy.

10.5 Why the Rule-Based Policy May Outperform DQN

A hand-written controller can outperform a learned DQN when the correct decision logic is simple and already known. The rule-based policy uses direct knowledge of the goal and controller target, so it can act correctly from the very first episode. DQN must approximate useful behaviour from sampled rewards and transitions, and its performance can be affected by exploration, approximation error, limited training, and the distribution of sampled goals. The value of DQN in this assignment is therefore not only raw performance, but also demonstrating learned high-level decision-making from interaction data.

11. Discussion of Failures, Stability, and Generalization

No complete failures were observed in the final evaluation because the selected DQN achieved 20 successes out of 20 episodes. The main failures were therefore concentrated in the early training stage, where both configurations experienced noisy rewards and a lower rolling success rate while the policy was still exploring. Configuration A, with slower decay, displayed a longer exploratory phase, while Configuration B transitioned more quickly to stable high success. Once learning matured, however, both configurations became fully reliable.

The plots suggest that the learned policy became highly stable after the initial exploration period. The reward moving average became positive and stable, the rolling success rate reached 1.00 and stayed there, and the greedy evaluation matched this strong performance. The larger-magnitude targets produced lower rewards than the smaller-magnitude targets, which suggests that longer movements remained more difficult, but not difficult enough to produce final evaluation failures.

Regarding oscillation, the numerical results do not show evidence of severe instability. The selected DQN ended with a very low final error and shorter mean episode length than the baseline. This implies that the policy learned to settle efficiently near the goal. The only remaining qualitative issue to monitor in the rendered viewer is whether the elbow occasionally makes a few final corrective adjustments before holding position. Even if that occurs, it does not materially reduce the performance of the learned controller in this experiment.

The exploration-decay schedule influenced how experience was collected in the replay buffer. Slower decay in Configuration A preserved exploration longer and may have contributed to its slightly better mean evaluation reward, while faster decay in Configuration B produced earlier stable exploitation and a shorter training time. Because both models achieved perfect evaluation success, the practical difference between the two settings was small.

12. Evidence-Based Recommendation

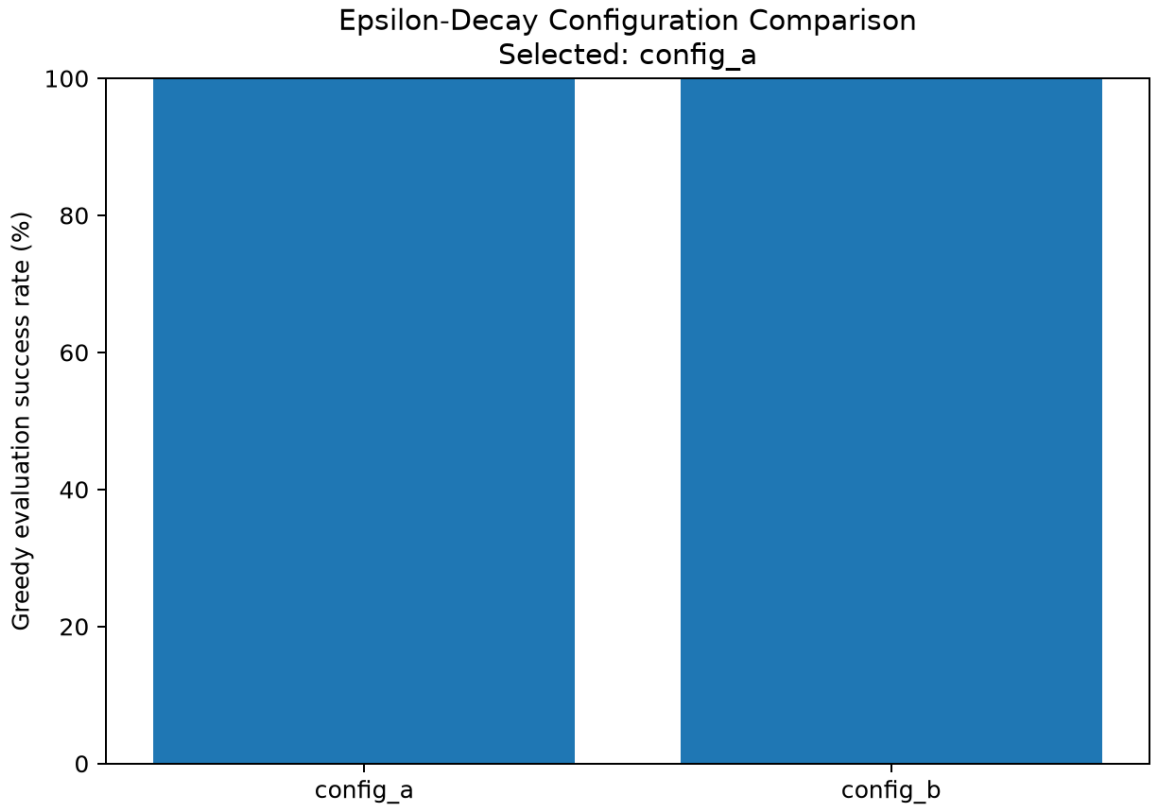

Figure 9. Epsilon-decay configuration comparison and selected model.

Based on the measured results, Configuration A is recommended as the stronger exploration-decay setting for the final submission. Both configurations achieved 100% success in the final 50 training episodes and 100% success during the 20-episode greedy evaluation. Configuration B trained faster and had a slightly higher mean reward over the final 20 training episodes, but

Configuration A achieved the higher mean evaluation reward of 13.2136 compared with 13.1641 for Configuration B. Since evaluation without exploration is the most important final criterion, Configuration A is the better overall choice.

The recommendation is therefore based on multiple pieces of evidence rather than one isolated metric: identical final success rates, slightly better evaluation reward for Configuration A, full consistency across all four target angles, and no stability concerns in the reported plots. The difference between the two models is small, which also suggests that both epsilon-decay settings were suitable for this relatively simple one-joint control task.

13. Limitations and Future Improvements

This experiment is limited to a one-joint control problem with a fixed-base robot. The learned policy does not solve full-body balance, locomotion, or coordinated multi-joint control. The parameter study also compares only two epsilon-decay settings while keeping the remaining baseline hyperparameters fixed.

Future work could evaluate additional random seeds to estimate performance variance, compare Double DQN to reduce Q-value overestimation, investigate prioritized experience replay, or test different network sizes. Other possible improvements include action hysteresis or reward shaping that encourages stable HOLD behaviour, although changes to the approved environment reward or success definition would require instructor approval for this assignment.

14. Conclusion

This assignment implemented a student-written PyTorch DQN for high-level discrete control of the Unitree G1 left elbow. The agent mapped a four-value continuous observation to three Q-values and learned through experience replay, epsilon-greedy exploration, Bellman updates, an online network, and a periodically synchronized target network.

Two epsilon-decay settings were compared under controlled conditions. Both configurations achieved 100% training success over the final 50 episodes and 100% greedy evaluation success over the required 20 benchmark episodes. Configuration A, using epsilon decay 0.995, was selected as the stronger final model because it achieved the slightly higher mean evaluation reward.

The selected DQN achieved 20 successful episodes out of 20, which exceeded the required 80% success threshold. It generalized successfully across all four benchmark goals. Compared with the supplied rule-based baseline, both policies achieved 100% success, but the DQN produced a higher mean cumulative reward, a shorter mean episode length, and a lower mean final absolute error. Overall, the experiment shows that DQN can learn an effective and accurate multi-goal

elbow-control policy from experience while also highlighting that a transparent rule-based policy can remain more sample efficient when strong task knowledge is already available.

Appendix A - Main Execution Commands

```
cd ~/Unitree_MuJoCo_G1_Primer_Workshop

source ~/.venvs/unitree/bin/activate

export PYTHONPATH="$PYTHONPATH:src"

python -m dqn.evaluate_rule_based --output-dir results/rule_based --seed 1000

python -m dqn.smoke_test

python -m dqn.train_dqn --config-name config_a --epsilon-decay 0.995 --
episodes 650 --seed 42 --device cpu --max-hours 2.30

python -m dqn.train_dqn --config-name config_b --epsilon-decay 0.985 --
episodes 650 --seed 42 --device cpu --max-hours 2.30

python -m dqn.evaluate_dqn --checkpoint models/config_a/best.pt --output-dir
results/config_a --seed 1000 --device cpu

python -m dqn.evaluate_dqn --checkpoint models/config_b/best.pt --output-dir
results/config_b --seed 1000 --device cpu

python -m dqn.compare_results

python -m dqn.compare_rule_based

python -m dqn.plot_results

python -m dqn.generate_report_summary

python -m dqn.render_dqn_policy --checkpoint models/selected_dqn.pt --goals -
0.8 -0.4 0.4 0.8 --device cpu
```

Appendix B - AI-Assistance Disclosure

AI tools were used to help organize and edit the written report for clarity and formatting. All code execution, training runs, evaluation runs, plots, screenshots, and numerical results reported in this assignment were generated from my own implementation and my own experiment outputs. I reviewed and edited the final submission before submission.